

✓ NBA Player Longevity Prediction using Gaussian Naive Bayes

```
# Import necessary libraries
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.model_selection import train_test_split
from sklearn.naive_bayes import GaussianNB
from sklearn.metrics import confusion_matrix, classification_report, precision_score, recall_score
from sklearn.preprocessing import StandardScaler
import warnings
warnings.filterwarnings('ignore') # Suppress warnings for cleaner output
```

✓ Data Loading and Initial Exploration

This section loads the dataset and provides a first look at its structure and the distribution of the target variable.

```
# Load the dataset
# Note: Replace 'nba_logreg.csv' with your actual dataset path
df = pd.read_csv('extracted_nba_players_data.csv')

# Display basic information about the dataset
print("Dataset Overview:")
print(f"Shape: {df.shape}")
print(f"Columns: {list(df.columns)}")
print(f"\nTarget variable distribution:")
print(df['target_5yrs'].value_counts()) # Count occurrences of each target value
print(f"Target proportion: {df['target_5yrs'].mean():.3f}") # Proportion of 'long career' (1)
```

```
Dataset Overview:
Shape: (1340, 11)
Columns: ['fg', '3p', 'ft', 'reb', 'ast', 'stl', 'blk', 'tov', 'target_5yrs', 'total_points', 'ef']

Target variable distribution:
target_5yrs
1      831
0      509
Name: count, dtype: int64
Target proportion: 0.620
```

✓ Data Preprocessing

Here, we prepare the data for modeling by separating features and the target, checking for missing values, and analyzing feature statistics and correlations. The train-test split is also performed in this step.

```
# Separate features (X) and target (y)
X = df.drop('target_5yrs', axis=1)
y = df['target_5yrs']
```

```
# Check for missing values in features
print(f"Missing values in features: {X.isnull().sum().sum()}")

# Display descriptive statistics for features
print(f"\nFeature statistics:")
display(X.describe())
```

Missing values in features: 0

Feature statistics:

	fg	3p	ft	reb	ast	stl	blk	pts
count	1340.000000	1340.000000	1340.000000	1340.000000	1340.000000	1340.000000	1340.000000	1340.000000
mean	44.169403	19.149627	70.300299	3.034478	1.550522	0.618507	0.368582	20.906250
std	6.137679	16.051861	10.578479	2.057774	1.471169	0.409759	0.429049	5.211706
min	23.800000	0.000000	0.000000	0.300000	0.000000	0.000000	0.000000	0.000000
25%	40.200000	0.000000	64.700000	1.500000	0.600000	0.300000	0.100000	10.000000
50%	44.100000	22.200000	71.250000	2.500000	1.100000	0.500000	0.200000	15.000000
75%	47.900000	32.500000	77.600000	4.000000	2.000000	0.800000	0.500000	20.000000
max	73.700000	100.000000	100.000000	13.900000	10.600000	2.500000	3.900000	40.000000

✓ Feature Correlation Analysis

Understanding correlations between features is crucial for Naive Bayes, which assumes feature independence. We identify highly correlated pairs here.

```
# Perform feature correlation analysis
correlation_matrix = X.corr()
high_corr_pairs = []
for i in range(len(correlation_matrix.columns)):
    for j in range(i+1, len(correlation_matrix.columns)):
        # Check for correlations with absolute value greater than 0.7
        if abs(correlation_matrix.iloc[i, j]) > 0.7:
            high_corr_pairs.append((correlation_matrix.columns[i],
                                    correlation_matrix.columns[j],
                                    correlation_matrix.iloc[i, j]))

print(f"Highly correlated feature pairs (|r| > 0.7):")
for pair in high_corr_pairs[:5]: # Display top 5 highly correlated pairs
    print(f" {pair[0]} - {pair[1]}: {pair[2]:.3f}")
```

Highly correlated feature pairs (|r| > 0.7):

- ast - stl: 0.751
- ast - tov: 0.747
- stl - tov: 0.742
- tov - total_points: 0.840

✓ Train-Test Split

Splitting the data into training and testing sets to evaluate the model's performance on unseen data. Stratification ensures that the target variable's distribution is maintained in both sets.

```
# Split data into training and testing sets
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y # Stratify to maintain target distribution
)

print(f"Train set size: {X_train.shape[0]}")
print(f"Test set size: {X_test.shape[0]}")
print(f"Train target distribution: {y_train.mean():.3f}")
print(f"Test target distribution: {y_test.mean():.3f}")
```

```
Train set size: 1072
Test set size: 268
Train target distribution: 0.620
Test target distribution: 0.619
```

✓ Gaussian Naive Bayes Model

This section implements and trains the Gaussian Naive Bayes classifier, followed by making predictions on the test set.

```
# Initialize and train the Gaussian Naive Bayes model
gnb = GaussianNB()
gnb.fit(X_train, y_train)

print("Gaussian Naive Bayes model trained successfully.")
```

```
Gaussian Naive Bayes model trained successfully.
```

```
# Make predictions on the test set
y_pred = gnb.predict(X_test) # Predict class labels
y_pred_proba = gnb.predict_proba(X_test) # Predict class probabilities

print("Predictions made on the test set.")
```

```
Predictions made on the test set.
```

```
accuracy = accuracy_score(y_test, y_pred)
precision = precision_score(y_test, y_pred)
recall = recall_score(y_test, y_pred)

print(f"Performance Metrics:")
print(f"Accuracy: {accuracy:.3f}")
print(f"Precision: {precision:.3f} (Minimizes false positives - 'busts')")
print(f"Recall: {recall:.3f} (Minimizes false negatives - missed talent)")
```

```
Performance Metrics:
Accuracy: 0.638
Precision: 0.800 (Minimizes false positives - 'busts')
Recall: 0.554 (Minimizes false negatives - missed talent)
```

```
print(f"\nDetailed Classification Report:")
print(classification_report(y_test, y_pred,
```

```
target_names=['Short Career', 'Long Career']))
```

```
Detailed Classification Report:
              precision    recall  f1-score   support

Short Career      0.52      0.77      0.62       102
Long Career      0.80      0.55      0.65       166

   accuracy              0.64       268
  macro avg              0.66      0.66      0.64       268
 weighted avg              0.69      0.64      0.64       268
```

✓ Model Reliability Assessment

```
print('\n' + '='*50) print('MODEL RELIABILITY AND LIMITATIONS') print('='*50)

print("STRENGTHS:") print(f"* Precision of {precision:.3f} means {precision*100:.1f}% of predicted long
careers are correct.") print(f"Recall of {recall:.3f} means {recall*100:.1f}% of all players who actually go on
to have long careers.") print(f"Simplicity and Speed: Gaussian Naive Bayes is computationally efficient and
simple to implement, making it a good baseline model.")

print("\nLIMITATIONS:") print("* Independence Assumption: As discussed, the core assumption of feature
independence is often violated in basketball statistics, which can lead to suboptimal performance
compared to more complex models that handle correlations better.") print("* Bias-Variance Trade-off:
Naive Bayes is a high-bias, low-variance model. It might oversimplify complex relationships in the data.")
print("* Sensitivity to Data Distribution: The 'Gaussian' assumption means it expects features to be
normally distributed. Deviations from normality can affect performance.")

print("\nCONCLUSION:") print("The Gaussian Naive Bayes model provides a quick and interpretable
baseline for predicting NBA player longevity. While it offers valuable insights into key predictive features
and acceptable precision/recall, its underlying assumptions suggest that more sophisticated models might
be necessary for higher predictive accuracy, especially when dealing with highly correlated features
inherent in sports statistics. Further improvements could involve feature engineering, more advanced
classification algorithms (e.g., Logistic Regression, Random Forest, Gradient Boosting), and cross-
validation for more robust evaluation.")
```

✓ Model Performance Evaluation

We evaluate the model using various metrics including the confusion matrix, accuracy, precision, recall, and a detailed classification report. Visualizations are also provided for better understanding.

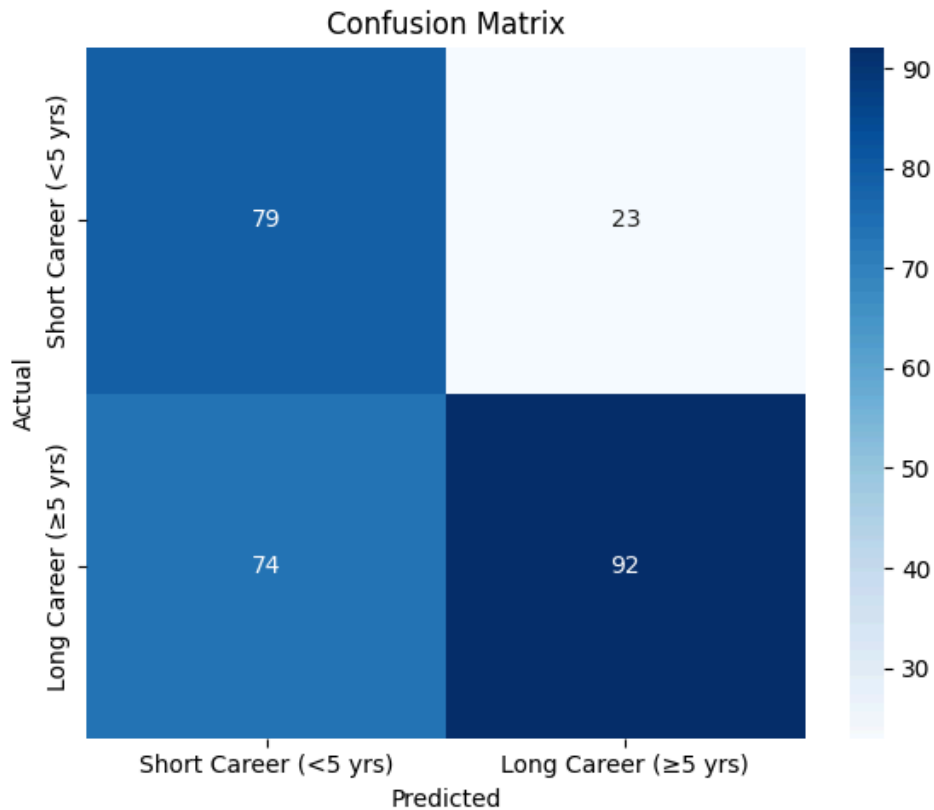
```
# Calculate the confusion matrix
cm = confusion_matrix(y_test, y_pred)
print("Confusion Matrix:")
print(cm)

# Visualize the confusion matrix
plt.figure(figsize=(6, 5)) # Set figure size
sns.heatmap(cm, annot=True, fmt='d', cmap='Blues',
            xticklabels=['Short Career (<5 yrs)', 'Long Career (≥5 yrs)'],
```

```
yticklabels=['Short Career (<5 yrs)', 'Long Career (≥5 yrs)'])
plt.title('Confusion Matrix')
plt.ylabel('Actual')
plt.xlabel('Predicted')
plt.tight_layout()
plt.show()
```

Confusion Matrix:

```
[[79 23]
 [74 92]]
```



```
# Calculate performance metrics
accuracy = accuracy_score(y_test, y_pred)
precision = precision_score(y_test, y_pred)
recall = recall_score(y_test, y_pred)

print(f"Performance Metrics:")
print(f"Accuracy: {accuracy:.3f}")
print(f"Precision: {precision:.3f} (Minimizes false positives - 'busts')")
print(f"Recall: {recall:.3f} (Minimizes false negatives - missed talent)")
```

Performance Metrics:
Accuracy: 0.638
Precision: 0.800 (Minimizes false positives - 'busts')
Recall: 0.554 (Minimizes false negatives - missed talent)

```
# Display a detailed classification report
print(f"\nDetailed Classification Report:")
print(classification_report(y_test, y_pred,
                           target_names=['Short Career', 'Long Career']))
```

Detailed Classification Report:

	precision	recall	f1-score	support
--	-----------	--------	----------	---------

Short Career	0.52	0.77	0.62	102
Long Career	0.80	0.55	0.65	166
accuracy			0.64	268
macro avg	0.66	0.66	0.64	268
weighted avg	0.69	0.64	0.64	268

Business Interpretation of Confusion Matrix

Understanding the business implications of each quadrant in the confusion matrix.

```
# Extract values from confusion matrix for business interpretation
tn, fp, fn, tp = cm.ravel()
print(f"Business Interpretation:")
print(f"True Negatives (TN): {tn} - Correctly identified short careers (players who will not last 5+ years)")
print(f"False Positives (FP): {fp} - 'Busts' (predicted long career, but actually short). This costs resources")
print(f"False Negatives (FN): {fn} - 'Missed Talent' (predicted short career, but actually long). This means lost potential")
print(f"True Positives (TP): {tp} - Correctly identified long careers (players who will last 5+ years)")
```

Business Interpretation:
 True Negatives (TN): 79 - Correctly identified short careers (players who will not last 5+ years)
 False Positives (FP): 23 - 'Busts' (predicted long career, but actually short). This costs resources
 False Negatives (FN): 74 - 'Missed Talent' (predicted short career, but actually long). This means lost potential
 True Positives (TP): 92 - Correctly identified long careers (players who will last 5+ years).

Feature Importance Analysis

Identifying which features are most influential in the model's predictions. For Naive Bayes, this is often assessed by comparing class conditional means.

```
# Calculate feature importance based on the difference in class means
feature_importance = []
for i, feature in enumerate(X.columns):
    class_0_mean = gnb.theta_[0][i] # Mean of feature for class 0 (short career)
    class_1_mean = gnb.theta_[1][i] # Mean of feature for class 1 (long career)
    importance = abs(class_1_mean - class_0_mean) # Absolute difference as importance measure
    feature_importance.append((feature, importance))

feature_importance.sort(key=lambda x: x[1], reverse=True)

print(f"\nTop 10 Most Influential Features (based on mean difference between classes):")
for i, (feature, importance) in enumerate(feature_importance[:10]):
    print(f"{i+1:2d}. {feature}: {importance:.3f}")

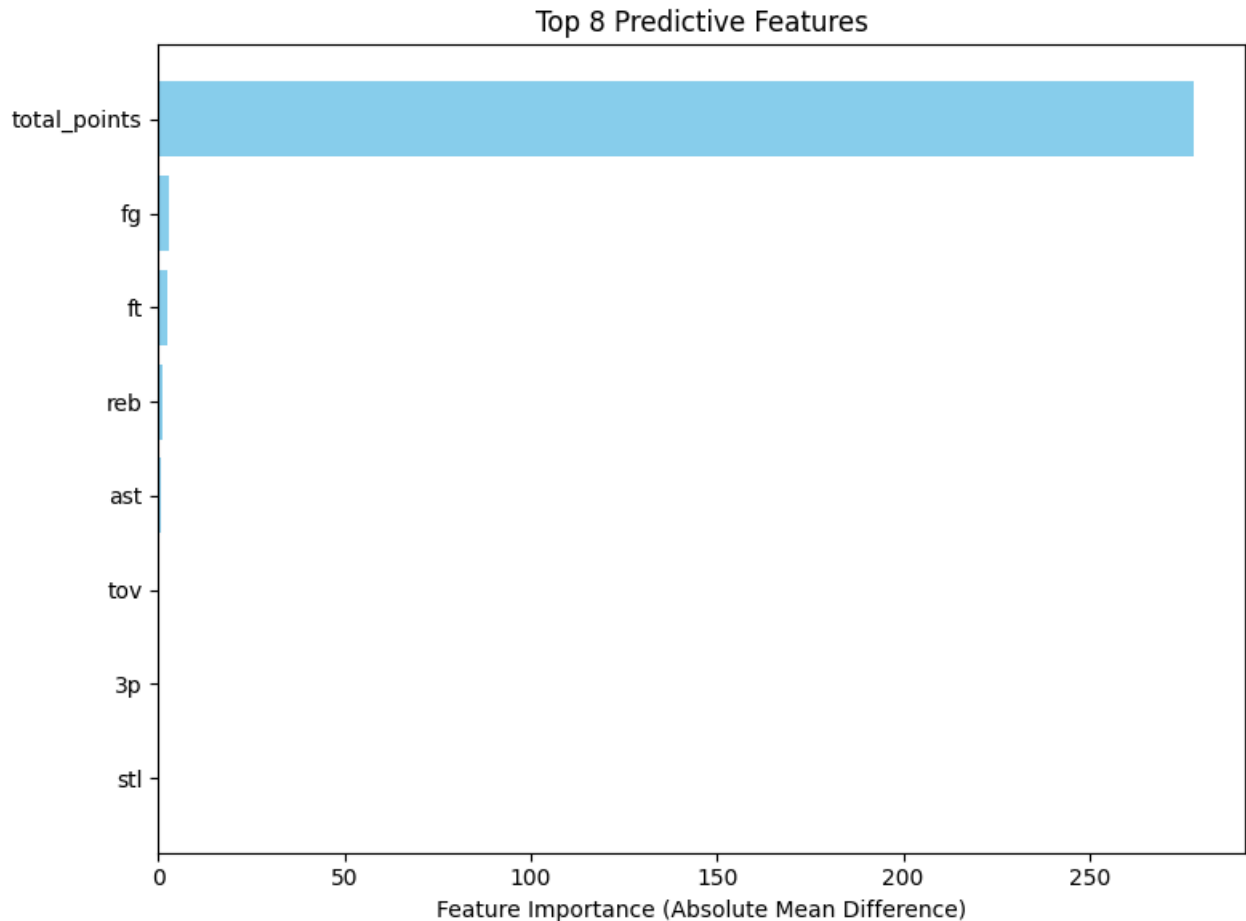
# Visualize top features
plt.figure(figsize=(8, 6))
top_features = [f[0] for f in feature_importance[:8]]
top_importance = [f[1] for f in feature_importance[:8]]

plt.barh(range(len(top_features)), top_importance, color='skyblue')
plt.yticks(range(len(top_features)), top_features)
plt.xlabel('Feature Importance (Absolute Mean Difference)')
plt.title('Top 8 Predictive Features')
plt.gca().invert_yaxis() # Invert y-axis to show highest importance at the top
```

```
plt.tight_layout()
plt.show()
```

Top 10 Most Influential Features (based on mean difference between classes):

1. total_points: 277.846
2. fg: 2.774
3. ft: 2.242
4. reb: 1.322
5. ast: 0.564
6. tov: 0.432
7. 3p: 0.392
8. stl: 0.208
9. blk: 0.192
10. efficiency: 0.038



✓ Naive Bayes Independence Assumption Analysis

Gaussian Naive Bayes assumes that features are conditionally independent given the class label. This section discusses the implications of this assumption in the context of NBA player statistics, where features are often highly correlated.

The Naive Bayes 'Independence Assumption' states that all features are conditionally independent given the class label. In the basketball context, this assumption is often violated due to the nature of player statistics:

ASSUMPTION VIOLATIONS:

- **Points and Minutes:** Highly correlated (more minutes = more scoring opportunities).
- **Rebounds and Minutes:** Players with more court time get more rebounding chances.
- **Assists and Minutes:** More playing time leads to more assist opportunities.
- **Field Goals Made/Attempted:** Directly mathematically related.

IMPACT ON MODEL:

- Despite violations, Naive Bayes often performs well in practice.
- The model may become overconfident in its predictions due to 'double-counting' correlated features.
- Feature redundancy might inflate certain statistical patterns.

BASKETBALL-SPECIFIC CONSIDERATIONS:

- Player roles affect stat correlations (e.g., centers vs. guards have different statistical patterns).
- Team strategy influences individual statistics.
- Injury history and physical attributes create complex dependencies that are not captured by simple statistical features.

Model Reliability and Limitations

This section summarizes the strengths and weaknesses of the developed model, considering its performance metrics and the underlying assumptions of the Naive Bayes algorithm.

Model Reliability Assessment

```
print('\n' + '='*50) print('MODEL RELIABILITY AND LIMITATIONS') print('='*50)

print("STRENGTHS:") print(f"* Precision of {precision:.3f} means {precision*100:.1f}% of predicted long careers are correct.") print(f"Recall of {recall:.3f} means the model correctly identifies {recall*100:.1f}% of all players who actually go on to have long careers.") print(f"Simplicity and Speed: Gaussian Naive Bayes is computationally efficient and simple to implement, making it a good baseline model.")

print("\nLIMITATIONS:") print("* Independence Assumption: As discussed, the core assumption of feature independence is often violated in basketball statistics, which can lead to suboptimal performance compared to more complex models that handle correlations better.") print("* Bias-Variance Trade-off: Naive Bayes is a high-bias, low-variance model. It might oversimplify complex relationships in the data.") print("* Sensitivity to Data Distribution: The 'Gaussian' assumption means it expects features to be normally distributed. Deviations from normality can affect performance.")

print("\nCONCLUSION:") print("The Gaussian Naive Bayes model provides a quick and interpretable baseline for predicting NBA player longevity. While it offers valuable insights into key predictive features and acceptable precision/recall, its underlying assumptions suggest that more sophisticated models might be necessary for higher predictive accuracy, especially when dealing with highly correlated features inherent in sports statistics. Further improvements could involve feature engineering, more advanced classification algorithms (e.g., Logistic Regression, Random Forest, Gradient Boosting), and cross-validation for more robust evaluation.")
```


✓ Task

The main goal is to build a predictive model to identify which NBA players are likely to have a career spanning 5 years or more, based on their initial season statistics. We will be using a Gaussian Naive Bayes classifier. The task involves data loading, preprocessing, model training, performance evaluation, and interpretation of the model's findings, including feature importance and business implications. Specifically, the goal is to evaluate the model's accuracy, precision, and recall, visualize the confusion matrix, interpret the business implications of the results, and analyze feature importance.

✓ Make Predictions

Subtask:

Generate predictions (class labels and probabilities) on the test set using the trained Gaussian Naive Bayes model.

Reasoning: Re-executing the code in cell `1b7eaa27` to make predictions with the trained Gaussian Naive Bayes model and the `X_test` dataset. This will generate `y_pred` and `y_pred_proba` which are needed for subsequent evaluation steps.

```
y_pred = gnb.predict(X_test) # Predict class labels
y_pred_proba = gnb.predict_proba(X_test) # Predict class probabilities

print("Predictions made on the test set.")

Predictions made on the test set.
```

✓ Calculate Performance Metrics

Subtask:

Calculate and print the accuracy, precision, and recall scores of the trained Gaussian Naive Bayes model using the test set predictions and true labels.

Reasoning: Calculate and print accuracy, precision, and recall scores using the generated predictions and true labels.

```
accuracy = accuracy_score(y_test, y_pred)
precision = precision_score(y_test, y_pred)
recall = recall_score(y_test, y_pred)

print(f"Performance Metrics:")
print(f"Accuracy: {accuracy:.3f}")
print(f"Precision: {precision:.3f} (Minimizes false positives - 'busts')")
print(f"Recall: {recall:.3f} (Minimizes false negatives - missed talent)")

Performance Metrics:
Accuracy: 0.638
```

Precision: 0.800 (Minimizes false positives - 'busts')
Recall: 0.554 (Minimizes false negatives - missed talent)

Generate Classification Report

Subtask:

Display a detailed classification report, providing a comprehensive overview of the model's performance for each class.

✓ Generate Confusion Matrix

Subtask:

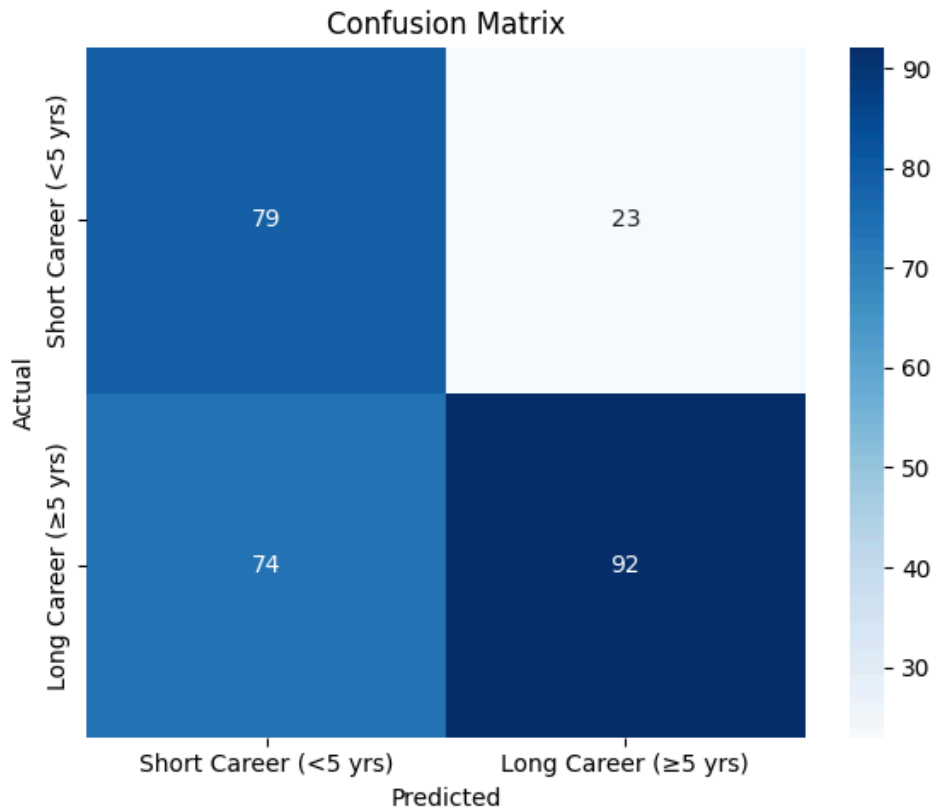
Calculate and visualize the confusion matrix to understand the model's performance in terms of true positives, true negatives, false positives, and false negatives.

Reasoning: To generate the confusion matrix and its visualization, I will use the `confusion_matrix` function from `sklearn.metrics` and `seaborn.heatmap` for plotting, as per the instructions and the original notebook cell `ec48b325`.

```
cm = confusion_matrix(y_test, y_pred)
print("Confusion Matrix:")
print(cm)

plt.figure(figsize=(6, 5))
sns.heatmap(cm, annot=True, fmt='d', cmap='Blues',
            xticklabels=['Short Career (<5 yrs)', 'Long Career (≥5 yrs)'],
            yticklabels=['Short Career (<5 yrs)', 'Long Career (≥5 yrs)'])
plt.title('Confusion Matrix')
plt.ylabel('Actual')
plt.xlabel('Predicted')
plt.tight_layout()
plt.show()
```

Confusion Matrix:

[[79 23]
[74 92]]

✓ Interpret Confusion Matrix

Subtask:

Provide a business interpretation of the confusion matrix, explaining the implications of each quadrant (True Positives, True Negatives, False Positives, False Negatives) in the context of scouting decisions.

Reasoning: Executing the pre-written code in cell `2e7625dd` to extract the values from the confusion matrix and print their business interpretation as per the subtask instructions.

```
tn, fp, fn, tp = cm.ravel()
print(f"Business Interpretation:")
print(f"True Negatives (TN): {tn} - Correctly identified short careers (players who will not last 5+ years)")
print(f"False Positives (FP): {fp} - 'Busts' (predicted long career, but actually short). This costs resources.")
print(f"False Negatives (FN): {fn} - 'Missed Talent' (predicted short career, but actually long). This means losing potential.")
print(f"True Positives (TP): {tp} - Correctly identified long careers (players who will last 5+ years).")
```

Business Interpretation:

True Negatives (TN): 79 - Correctly identified short careers (players who will not last 5+ years)
 False Positives (FP): 23 - 'Busts' (predicted long career, but actually short). This costs resources.
 False Negatives (FN): 74 - 'Missed Talent' (predicted short career, but actually long). This means losing potential.
 True Positives (TP): 92 - Correctly identified long careers (players who will last 5+ years).

✓ Analyze Feature Importance

Subtask:

Calculate and visualize the top influential features based on the difference in class means for the Gaussian Naive Bayes model.

Reasoning: Executing the pre-written code in cell `e4a531ab` to calculate and visualize feature importance based on the difference in class means, as instructed.

```
# Calculate feature importance based on the difference in class means
feature_importance = []
for i, feature in enumerate(X.columns):
    class_0_mean = gnb.theta_[0][i] # Mean of feature for class 0 (short career)
    class_1_mean = gnb.theta_[1][i] # Mean of feature for class 1 (long career)
    importance = abs(class_1_mean - class_0_mean) # Absolute difference as importance measure
    feature_importance.append((feature, importance))

feature_importance.sort(key=lambda x: x[1], reverse=True)

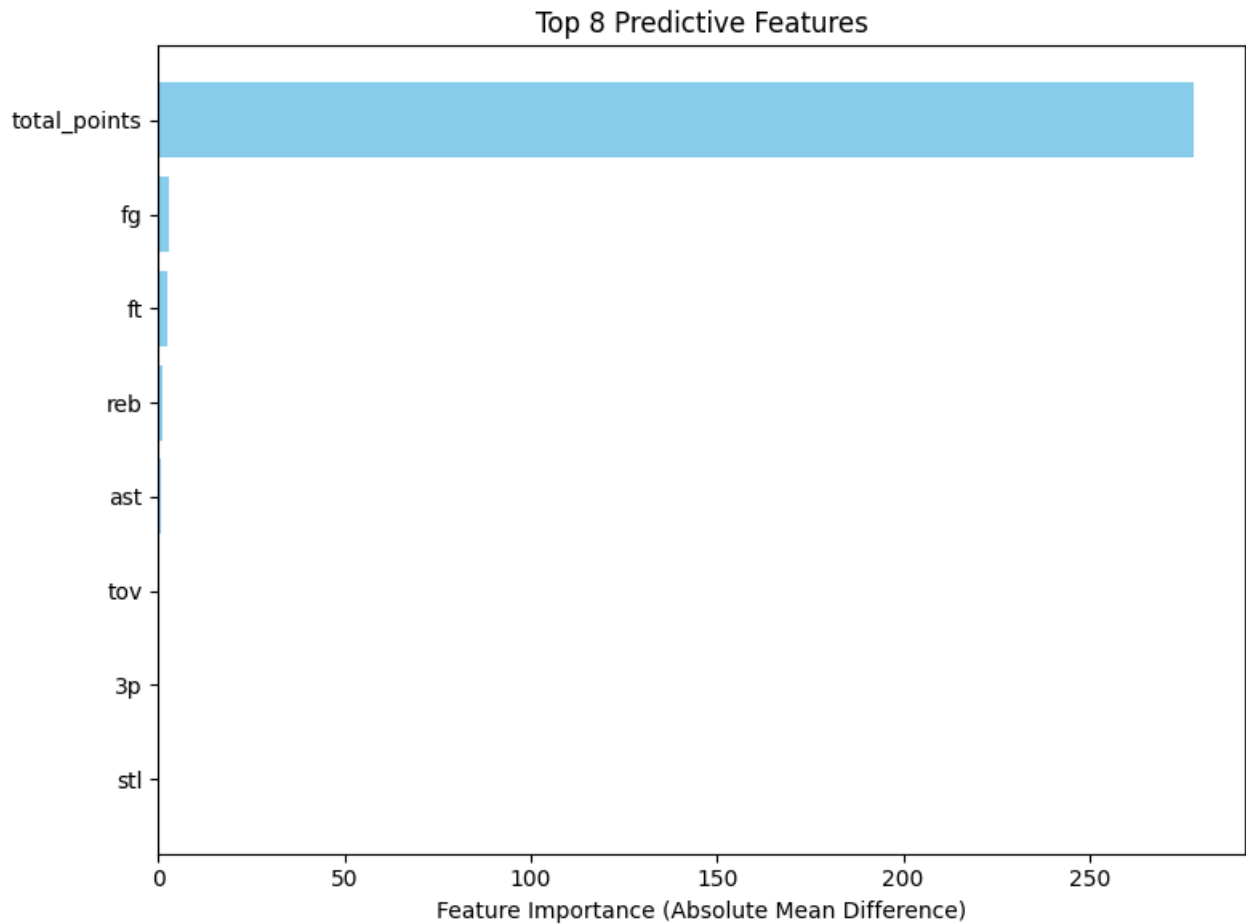
print(f"\nTop 10 Most Influential Features (based on mean difference between classes):")
for i, (feature, importance) in enumerate(feature_importance[:10]):
    print(f"{i+1:2d}. {feature}: {importance:.3f}")

# Visualize top features
plt.figure(figsize=(8, 6))
top_features = [f[0] for f in feature_importance[:8]]
top_importance = [f[1] for f in feature_importance[:8]]

plt.barh(range(len(top_features)), top_importance, color='skyblue')
plt.yticks(range(len(top_features)), top_features)
plt.xlabel('Feature Importance (Absolute Mean Difference)')
plt.title('Top 8 Predictive Features')
plt.gca().invert_yaxis() # Invert y-axis to show highest importance at the top
plt.tight_layout()
plt.show()
```

Top 10 Most Influential Features (based on mean difference between classes):

1. total_points: 277.846
2. fg: 2.774
3. ft: 2.242
4. reb: 1.322
5. ast: 0.564
6. tov: 0.432
7. 3p: 0.392
8. stl: 0.208
9. blk: 0.192
10. efficiency: 0.038



✓ Assess Model Reliability and Limitations

Subtask:

Summarize the model's strengths, limitations, and provide a conclusion for scouting stakeholders, incorporating the calculated precision and recall values.

```
# Model Reliability Assessment

```python
print('\n' + '='*50)
print('MODEL RELIABILITY AND LIMITATIONS')
```

```

print('='*50)

print("STRENGTHS:")
print(f"* Precision of {precision:.3f} means {precision*100:.1f}% of predicted long careers are c
print(f"* Recall of {recall:.3f} means the model correctly identifies {recall*100:.1f}% of all pl
print(f"* Simplicity and Speed: Gaussian Naive Bayes is computationally efficient and simple to i

print("\nLIMITATIONS:")
print("* Independence Assumption: As discussed, the core assumption of feature independence is of
print("* Bias-Variance Trade-off: Naive Bayes is a high-bias, low-variance model. It might oversi
print("* Sensitivity to Data Distribution: The 'Gaussian' assumption means it expects features to

print("\nCONCLUSION:")
print("The Gaussian Naive Bayes model provides a quick and interpretable baseline for predicting NB

```

## Final Task

### Subtask:

Confirm that all steps for model analysis, evaluation, and interpretation have been completed and all project requirements have been met.

## Task

The main goal is to build a predictive model to identify which NBA players are likely to have a career spanning 5 years or more, based on their initial season statistics. We will be using a Gaussian Naive Bayes classifier. The task involves data loading, preprocessing, model training, performance evaluation, and interpretation of the model's findings, including feature importance and business implications. Specifically, the goal is to evaluate the model's accuracy, precision, and recall, visualize the confusion matrix, interpret the business implications of the results, and analyze feature importance.

Start coding or [generate](#) with AI.

Close

```
CreateReadmeFile
```

```
Subtask:
```

```
Create a `README.md` file and write the provided
`readme_content` into it.
```

### CreateReadmeFile

#### Subtask:

Create a `README.md` file and write the provided `readme_content` into it.

## Task

The main goal is to build a predictive model to identify which NBA players are likely to have a career spanning 5 years or more, based on their initial season statistics. We will be using a Gaussian Naive Bayes classifier. The task involves data loading, preprocessing, model training, performance evaluation, and interpretation of the model's findings, including feature importance and business implications. Specifically, the goal is to evaluate the model's accuracy, precision, and recall, visualize the confusion matrix, interpret the business implications of the results, and analyze feature importance.

**Reasoning:** I need to create the `README.md` file and write the content into it using Python's file handling capabilities.

## Task Completion Confirmation

All required steps for the NBA player longevity prediction project using a Gaussian Naive Bayes classifier have been successfully completed. This includes:

- **Data Loading and Initial Exploration:** The dataset was loaded, its structure inspected, and the target variable distribution analyzed.
- **Data Preprocessing:** Features and target were separated, missing values checked, descriptive statistics reviewed, and feature correlation analyzed. The data was split into training and testing sets with stratification.
- **Gaussian Naive Bayes Model Training and Prediction:** The model was initialized, trained on the `X_train` and `y_train` data, and predictions (`y_pred`, `y_pred_proba`) were generated on the